typst-adt

`adt` is a Typst package for describing algebraic data types and generic type wrappers over builtin types. From the description of a type specification (hereinafter referred to as a "spec"), `adt` generates corresponding validation functions, constructors, eliminators, field accessors, recursive folds, and annotation passes.

It has the following properties:

- First-class: specs are regular Typst values, functors are regular Typst functions from specs to specs;
- Bootstrapped: the specs themselves are described by an algebraic datatype and are able to be validated as well as matched on;
- Non-intrusive: besides sum types (`enum`s) which have a designated tag field, the values need not to carry any sort of type information;
- Non-infectious: as a consequence of the above, neither dependencies nor dependents need conversion.

Specs

`adt` bootstraps its type system and provides an algebraic datatype representing type specifications. Here it is for reference, which you can simultaneously think of as the example and the concept:

```
#let SPEC-NAME = adt.union(str, type(auto))

#let ARGS-SPEC(SPEC) = adt.enum(
  __name__: "args-spec(" + adt.to-string(SPEC) + ")",
  null: none,
  args: (pos: adt.array(SPEC), named: adt.dict(str, SPEC)),
)

#let CONSTR-SPEC(SPEC) = adt.enum(
  __name__: "constr-spec(" + adt.to-string(SPEC) + ")",
  null: none,
  fields: (fields: adt.dict(str, SPEC)),
)

#let SPEC = adt.fix(
  __name__: "spec",
  self => adt.enum(
    __name__: "spec-shape",
    empty: none,
    builtin: (name: SPEC-NAME, value: type),
    any: none,
    enum: (name: SPEC-NAME, constrs: adt.dict(str, CONSTR-SPEC(self))),
    struct: (name: SPEC-NAME, fields: adt.dict(str, self)),
    union: (name: SPEC-NAME, elems: adt.array(self)),
    array: (name: SPEC-NAME, inner: self),
    dict: (name: SPEC-NAME, key: self, value: self),
    function: (name: SPEC-NAME, dom: ARGS-SPEC(self), cod: self),
    fix: (name: str, fun: adt.fun(self)(self)),
    self: (depth: int),
  ),
)
```

Builtins

Use Typst builtin types directly:

```
#assert.eq(adt.validate(int, 4), ok(4))
#assert(result-is-err(adt.validate(int, "4")))
```

Use `adt.any` when any value is valid, and `adt.empty` when no value is valid.

Structs

Use `adt.struct` for dictionaries with fixed fields.

```
#let STYLE = adt.struct(__name__: "STYLE", size: length, color: color)
#let (intro: style, elim: style-elim) = adt.generate(STYLE)
```

```
#assert.eq(
  style(12pt, red),
  (size: 12pt, color: red)
)
#assert.eq(
  style-elim((size, color) => style(
    size + 8pt,
    color
  ))(style(12pt, red)),
  style(20pt, red)
)
```

Enums
Use `adt.enum` for tagged variants.

```
#let TOKEN = adt.enum(
  __name__: "TOKEN",
  eof: none,
  lit: int,
  span: (start: int, end: int),
)

#let (
  intro: (
    eof: token-eof,
    lit: token-lit,
    span: token-span,
  ),
  elim: token-elim,
) = adt.generate(TOKEN)

#let token-kind = token-elim(
  eof: "eof",
  lit: value => "lit:" + str(value),
  span: (start, end) => "span:" + str(end - start),
)

#assert.eq(token-kind(token-eof), "eof")
#assert.eq(token-kind(token-lit(9)), "lit:9")
#assert.eq(token-kind(token-span(3, 8)), "span:5")
```

Constructor shapes:

- `none`: no fields.
- `int`: one field named `value`.
- `(start: int, end: int)`: named fields.

Unions
Use `adt.union` when a value may match any of several specs.

```
#let INT-OR-STR = adt.union(int, str)
#let (intro: int-or-str, elim: int-or-str-elim) = adt.generate(INT-OR-STR)

#assert.eq(int-or-str(4), 4)
#assert.eq(int-or-str("ok"), "ok")
#assert.eq(int-or-str-elim(value => str(value))(4), "4")
#assert(result-is-err(adt.validate(INT-OR-STR, 1pt)))
```

Nested unions are flattened. `adt.union()` creates `adt.empty`.

Arrays
Use `adt.array(inner)` for arrays.

```
#let INTS = adt.array(int)
#let (intro: ints, elim: ints-elim) = adt.generate(INTS)

#assert.eq(ints((1, 2, 3)), (1, 2, 3))
#assert.eq(ints-elim(xs => xs.len())(ints((1, 2, 3))), 3)
#assert(result-is-err(adt.validate(INTS, (1, "bad"))))
```

Dictionaries
Use `adt.dict(key, value)` for dictionaries.

```
#let STR-INTS = adt.dict(str, int)
#let (intro: str-ints, elim: str-ints-elim) = adt.generate(STR-INTS)

#assert.eq(str-ints((a: 1, b: 2)).a, 1)
#assert.eq(str-ints-elim(xs => xs.len())(str-ints((a: 1, b: 2))), 2)
#assert(result-is-err(adt.validate(STR-INTS, (a: "bad"))))
```

Functions
Use `adt.fun(..domain)(codomain)` for functions.

```
#let ADD = adt.fun(int, int)(int)
#let (intro: add, elim: apply-add) = adt.generate(ADD)

#assert.eq(add((x, y) => x + y)(2, 3), 5)
#assert.eq(apply-add(add((x, y) => x * y))(4, 5), 20)
```

Named arguments are supported:

```
#let NAMED = adt.fun(left: int, right: int)(int)
#let (intro: named-add) = adt.generate(NAMED)

#assert.eq(named-add((left: 0, right: 0) => left + right)(
  left: 2,
  right: 3,
), 5)
```

Recursive specs

Use `adt.fix` for recursive data.

```
#let LIST(T) = adt.fix(
  __name__: "list(" + adt.to-string(T) + ")",
  self => adt.enum(
    nil: none,
    cons: (head: T, tail: self),
  ),
)

#let (
  intro: (
    nil: list-nil,
    cons: list-cons,
  ),
  fields: (
    head: list-head,
    tail: list-tail,
  ),
  elim: list-elim,
  rec: list-rec,
  annotate: list-annotate,
) = adt.generate(LIST(int))

#let list(..args) = {
  let xs = list-nil
  for elem in args.pos().rev() {
```

```
    xs = list-cons(elem, xs)
  }
  xs
}

#let one-two = list(1, 2)

#assert.eq(list-head(one-two), 1)
#assert.eq(list-tail(one-two).head, 2)

#let length = list-rec(
  nil: 0,
  cons: (head, tail-len) => tail-len + 1,
)

#assert.eq(length(one-two), 2)
```

Generated values are plain data. Use generated helpers explicitly:

```
#assert.eq(list-elim(
  nil: none,
  cons: (head, tail) => head,
)(one-two), 1)

#assert.eq(list-rec(
  nil: 0,
  cons: (head, tail-len) => tail-len + 1,
)(one-two), 2)
```

Use `adt.rec(spec, ..cases)` as a direct recursive fold helper when you do not need the rest of `adt.generate(spec)`.

Annotations

Recursive enum values can be annotated in one pass.

```
#let SIZED-LIST = adt.annotate(LIST(int), size: int)

#let sized = list-annotate(
  __ann__: (size: int),
  nil: (size: 0),
  cons: (head, tail) => (size: tail.size + 1),
)(list(1, 2))

#assert.eq(sized.size, 2)
#assert.eq(sized.tail.size, 1)
#assert.eq(adt.validate(SIZED-LIST, sized), ok(sized))
```

Results

Results are plain dictionaries tagged as `result/ok` or `result/err`. They are working result types so feel free to use them, but their purpose is to help with the internals of validation.

```
#let good = ok(4)
#let bad = err("expected int")

#assert.eq(good.value, 4)
#assert(result-is-ok(good))
#assert(result-is-err(bad))
```

Useful helpers:

- `adt.result-map(f, result)`
- `adt.result-map2(f, a, b)`
- `adt.result-and-then(result, cont)`

- `adt.result-all(f, xs)`
- `adt.result-all-dict(f, xs)`
- `adt.result-any(f, xs)`
- `adt.result-unwrap(result)`

Reference

Specs

- ARGS-SPEC()
- CONSTR-SPEC()
- annotate()
- args-spec-fields-aux()
- array()
- builtin()
- constr-spec-fields-aux()
- dict()
- enum()
- fix()
- fun()
- struct()
- union()
- union-elems()
- union2()

Variables

- args-spec-null
- constr-spec-null
- any
- empty
- SPEC-NAME
- SPEC

ARGS-SPEC
Spec for function argument specs parameterized by an inner spec.

Parameters
`ARGS-SPEC(T: spec) -> spec`

> **T** `spec`
>
> Spec used for argument entries.

CONSTR-SPEC
Spec for constructor specs parameterized by an inner spec.

Parameters
`CONSTR-SPEC(T: spec) -> spec`

> **T** `spec`
>
> Spec used for field entries.

annotate
Adds annotation fields to a spec.

For enum specs, every constructor receives the annotation fields. For fixed-point specs, recursive children are annotated too.

Parameters

```
annotate(
  spec: spec,
  ..ann: arguments
) -> spec
```

spec `spec`

Spec to annotate.

..ann `arguments`

Annotation field specs.

args-spec-fields-aux
Builds an argument spec from positional and named specs.

This is an internal constructor used after parsing shorthand argument specs.

Parameters

```
args-spec-fields-aux(
  spec-parse: function,
  pos: array(spec),
  named: dictionary(str, spec)
) -> args-spec
```

spec-parse `function`

Spec parser to apply recursively.

pos `array(spec)`

Positional argument specs.

named `dictionary(str, spec)`

Named argument specs.

array
Builds an array spec.

- `inner`: Spec for each array item.

Parameters

```
array(
  __name__: str auto,
  inner: spec
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

inner `spec`

Item spec.

builtin
Builds a spec for a builtin Typst type.

- `type_`: A Typst type such as `int`, `str`, or `dictionary`.

Parameters

```
builtin(type_: type) -> spec
```

type_ `type`

Builtin Typst type.

constr-spec-fields-aux
Builds a constructor spec from named field specs.

This is an internal constructor used after parsing shorthand constructor specs.

Parameters

```
constr-spec-fields-aux(
  spec-parse: function,
  fields: dictionary(str, spec)
) -> constr-spec
```

spec-parse `function`

Spec parser to apply recursively.

fields `dictionary(str, spec)`

Constructor field specs.

dict
Builds a dictionary spec.

- `key`: Spec for each key.
- `value`: Spec for each value.

Parameters

```
dict(
  __name__: str auto,
  key: spec,
  value: spec
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

key `spec`

Key spec.

value `spec`

Value spec.

enum
Builds an enum spec from named constructors.

Constructors are passed as named arguments. Use `none` for a nullary constructor, a spec for a single `value` field, or a dictionary/argument value for named fields.

Parameters

```
enum(
  __name__: str auto,
  ..args: arguments
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

..args `arguments`

Named constructor specs.

fix
Builds a recursive spec as a fixed point.

- `fun`: Function from the recursive self spec to the unfolded base spec.

Parameters

```
fix(
  __name__: str auto,
  fun: function
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

fun `function`

Function from self spec to unfolded spec.

fun

Builds a function spec from domain arguments and a codomain.

Positional and named domain arguments are passed first. The returned function accepts the codomain spec.

Parameters

```
fun(..dom: arguments) -> function
```

..dom `arguments`

Domain argument specs.

struct

Builds a struct spec from named fields.

Fields are passed as named arguments and parsed as specs.

Parameters

```
struct(
  __name__: str auto,
  ..args: arguments
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

..args `arguments`

Named field specs.

union
Builds a union spec from zero or more specs.

Nested unions are flattened. Calling with no specs returns `empty`.

Parameters

```
union(
  __name__: str auto,
  ..args: arguments
) -> spec
```

__name__ `str` or `auto`

Optional display name.

Default: `auto`

..args `arguments`

Specs to union.

union-elems
Returns the flattened elements of a union spec.

Non-union specs are returned as a one-element array.

Parameters

```
union-elems(spec: spec ) -> array(spec)
```

spec `spec`

Spec to inspect.

union2
Combines two specs into one flattened union.

Empty unions collapse away and one-element unions collapse to that element.

Parameters

```
union2(
  left: spec,
  right: spec
) -> spec
```

left `spec`

Left spec.

right `spec`

Right spec.

args-spec-null `args-spec`

Argument spec for functions that take no arguments.

constr-spec-null `constr-spec`

Constructor spec for constructors with no fields.

any `spec`

Spec that accepts any value.

empty `spec`

Spec that accepts no values.

SPEC-NAME `spec`

Spec for names used by specs.

SPEC `spec`

Meta-spec describing valid specs.

Useful for validating that a value is itself a well-formed spec.

Validation

- validate()
- validate-args()
- validate-args-aux()
- validate-constr()
- validate-constr-aux()

validate
Validates a value against a spec.

Returns `ok(validated-value)` when the value matches the spec, or `err(message)` otherwise.

Parameters

```
validate(
  spec: spec,
  value: any
) -> RESULT(any)
```

spec `spec`

Spec to validate against.

value `any`

Value to validate.

validate-args
Validates function call arguments with the default validator.

Returns validated Typst `arguments`.

Parameters

```
validate-args(
  args-spec: args-spec,
  ..args: arguments
) -> RESULT(arguments)
```

args-spec `args-spec`

Argument spec.

..args `arguments`

Arguments to validate.

validate-args-aux
Validates function call arguments against an argument spec.

This lower-level variant accepts the recursive validator to use.

Parameters

```
validate-args-aux(
  validate: function,
  args-spec: args-spec,
  ..args: arguments
) -> RESULT(arguments)
```

validate `function`

Validator to use recursively.

args-spec `args-spec`

Argument spec.

..args `arguments`

Arguments to validate.

validate-constr
Validates constructor arguments with the default validator.

Returns a dictionary of validated fields.

Parameters

```
validate-constr(
  constr-spec: constr-spec,
  ..args: arguments
) -> RESULT(dictionary)
```

constr-spec `constr-spec`

Constructor spec.

..args `arguments`

Constructor arguments to validate.

validate-constr-aux
Validates constructor arguments against a constructor spec.

This lower-level variant accepts the recursive validator to use.

Parameters

```
validate-constr-aux(
  validate: function,
  constr-spec: constr-spec,
  ..args: arguments
) -> RESULT(dictionary)
```

validate `function`

Validator to use recursively.

constr-spec `constr-spec`

Constructor spec.

..args `arguments`

Constructor arguments to validate.

Generation

- generate()

generate
Generates all helpers available for a spec.

The returned dictionary may contain `intro`, `intros`, `elim`, `fields`, `rec`, and `annotate`, depending on the spec kind.

Parameters

```
generate(spec) -> dictionary
```

Results

RESULT

Spec for result values.

A `RESULT(T)` is either `ok(value)` where `value` matches `T`, or `err(msg)` where `msg` is a string.

Parameters

```
RESULT(T: spec) -> spec
```

T	`spec`

Spec for the successful value.

err

Creates a failed result.

- `e`: Error message. Converted to a string.

Parameters

```
err(e: any) -> RESULT(any)
```

e	`any`

Error message.

ok

Creates a successful result.

- `x`: Value to store.

Parameters

```
ok(x: any) -> RESULT(any)
```

x `any`

Value to store.

result-all
Maps an array with a result-producing function.

Returns `ok(values)` when all items are ok. Returns one error containing all indexed failures otherwise.

Parameters

```
result-all(
  f: function,
  xs: array
) -> RESULT(array)
```

f `function`

Function returning a result for each item.

xs `array`

Items to map.

result-all-dict
Maps dictionary values with a result-producing function.

Keys are preserved. Returns the first aggregated error from `result-all`.

Parameters

```
result-all-dict(
  f: function,
  xs: dictionary
) -> RESULT(dictionary)
```

f `function`

Function returning a result for each value.

xs `dictionary`

Dictionary to map.

result-and-then
Chains a result into a result-producing continuation.

Calls `cont(value)` for ok results and passes errors through unchanged.

Parameters

```
result-and-then(
  result: RESULT(any),
  cont: function
) -> RESULT(any)
```

result `RESULT(any)`

Input result.

cont `function`

Continuation returning a result.

result-any
Returns the first ok result from mapping an array.

If every item fails, returns one error containing all failures.

Parameters

```
result-any(
  f: function,
  xs: array
) -> RESULT(any)
```

f `function`

Function returning a result for each item.

xs `array`

Items to try.

result-elim
Pattern matches a result.

Pass exactly two named cases: `ok:` and `err:`. Non-function cases are treated as constants.

Parameters

```
result-elim(..args: arguments) -> function
```

..args `arguments`

Named `ok` and `err` cases.

result-is-err
Returns whether `result` is an err result.

Parameters

```
result-is-err(result: RESULT(any))
```

result `RESULT(any)`

Result to test.

result-is-ok

Returns whether `result` is an ok result.

Parameters

```
result-is-ok(result: RESULT(any))
```

result `RESULT(any)`

Result to test.

result-map

Maps an ok result.

- `f`: Function applied to the ok value.
- `result`: Result to map.

Parameters

```
result-map(
  f: function,
  result: RESULT(any)
) -> RESULT(any)
```

f `function`

Function applied to the ok value.

result `RESULT(any)`

Result to map.

result-map2

Maps two ok results.

Calls `f(value1, value2)` only when both inputs are ok. The first error is returned unchanged.

Parameters

```
result-map2(
  f: function,
  result1: RESULT(any),
  result2: RESULT(any)
) -> RESULT(any)
```

f `function`

Function applied to both ok values.

result1 `RESULT(any)`

First result.

result2 `RESULT(any)`

Second result.

result-or-else
Returns `result1` if it is ok, otherwise returns `result2`.

Parameters

```
result-or-else(
  result1: RESULT(any),
  result2: RESULT(any)
) -> RESULT(any)
```

result1 `RESULT(any)`

Preferred result.

result2 `RESULT(any)`

Fallback result.

result-unwrap
Extracts the ok value.

Panics with the error message when given an err result.

Parameters

```
result-unwrap(result: RESULT(any)) -> any
```

result `RESULT(any)`

Result to unwrap.

result-validate
Validates an ok result value.

Err results pass through unchanged.

Parameters

```
result-validate(
  result: RESULT(any),
  validate: function
) -> RESULT(any)
```

result `RESULT(any)`

Result to validate.

validate `function`

Validator for the ok value.

result-zip
Zips two arrays with a result-producing function.

Returns an error on arity mismatch.

Parameters

```
result-zip(
  f: function,
  xs: array,
  ys: array
) -> RESULT(array)
```

f `function`

Function called with paired values.

xs `array`

First array.

ys `array`

Second array.

result-zip-dict
Zips matching dictionary keys with a result-producing function.

Returns an error when either dictionary is missing keys from the other.

Parameters

```
result-zip-dict(
  f: function,
  xs: dictionary,
  ys: dictionary
) -> RESULT(dictionary)
```

f `function`

Function called with paired values.

xs `dictionary`

First dictionary.

ys `dictionary`

Second dictionary.

Parsing

- args-spec-elim()
- args-spec-parse()
- args-spec-parse-aux()
- args-spec-to-string()
- args-spec-to-string-aux()
- constr-spec-elim()
- constr-spec-parse()
- constr-spec-parse-aux()
- constr-spec-to-string-aux()
- spec-elim()
- spec-parse()
- to-string()

args-spec-elim
Dispatches on an argument spec.

Parameters

```
args-spec-elim(
  null: function any auto,
  args: function any auto
) -> function
```

null `function` or `any` or `auto`

Case for no arguments.

Default: `auto`

args `function` or `any` or `auto`

Case for positional and named arguments.

Default: `auto`

args-spec-parse
Parses an argument spec using the default spec parser.

Parameters

```
args-spec-parse(args-spec: any) -> RESULT(args-spec)
```

args-spec `any`

Argument spec shorthand.

args-spec-parse-aux
Parses shorthand argument specs into explicit argument specs.

This lower-level variant accepts the recursive spec parser to use.

Parameters

```
args-spec-parse-aux(
  spec-parse: function,
  args-spec: any
) -> RESULT(args-spec)
```

spec-parse `function`

Spec parser to use recursively.

args-spec `any`

Argument spec shorthand.

args-spec-to-string
Renders an argument spec as a compact string.

Parameters

```
args-spec-to-string(args-spec: args-spec) -> str
```

args-spec `args-spec`

Argument spec.

args-spec-to-string-aux
Renders an argument spec with a custom spec renderer.

Used by `args-spec-to-string` and `spec-to-string`.

Parameters

```
args-spec-to-string-aux(
  spec-to-string: function,
  args-spec: args-spec
) -> str
```

spec-to-string `function`

Spec renderer.

args-spec `args-spec`

Argument spec.

constr-spec-elim
Dispatches on a constructor spec.

Pass `null:` and `fields:` cases. Cases may be functions or constants.

Parameters

```
constr-spec-elim(
  null: function any auto,
  fields: function any auto
) -> function
```

null `function` or `any` or `auto`

Case for constructors with no fields.

Default: `auto`

fields `function` or `any` or `auto`

Case for constructors with fields.

Default: `auto`

constr-spec-parse
Parses a constructor spec using the default spec parser.

Parameters

```
constr-spec-parse(constr-spec: any) -> RESULT(constr-spec)
```

constr-spec `any`

Constructor spec shorthand.

constr-spec-parse-aux
Parses shorthand constructor specs into explicit constructor specs.

This lower-level variant accepts the recursive spec parser to use.

Parameters

```
constr-spec-parse-aux(
  spec-parse: function,
  constr-spec: any
) -> RESULT(constr-spec)
```

spec-parse `function`

Spec parser to use recursively.

constr-spec `any`

Constructor spec shorthand.

constr-spec-to-string-aux
Renders a constructor spec with a custom spec renderer.

Used by `spec-to-string` for enum constructors.

Parameters

```
constr-spec-to-string-aux(
  spec-to-string: function,
  constr-spec: constr-spec
) -> str
```

spec-to-string `function`

Spec renderer.

constr-spec `constr-spec`

Constructor spec.

spec-elim
Dispatches on a spec kind.

All spec cases must be provided. This is the central eliminator for spec values.

Parameters

```
spec-elim(
  empty_case: function,
  builtin: function,
  any: function,
  union_case: function,
  enum: function,
  struct: function,
  array: function,
  dict: function,
  function: function,
  fix: function,
  self: function
) -> function
```

empty_case `function`

Case for `adt.empty`.

Default: `auto`

builtin `function`

Case for builtin specs.

Default: `auto`

any `function`

Case for `adt.any`.

Default: `auto`

union_case `function`

Case for union specs.

Default: `auto`

enum `function`

Case for enum specs.

Default: `auto`

struct `function`

Case for struct specs.

Default: `auto`

array `function`

Case for array specs.

Default: `auto`

dict `function`

Case for dictionary specs.

Default: `auto`

function `function`

Case for function specs.

Default: `auto`

fix `function`

Case for fixed-point specs.

Default: `auto`

self `function`

Case for recursive self refs.

Default: `auto`

spec-parse

Parses shorthand specs into explicit specs.

Builtin types become builtin specs, plain dictionaries become struct specs, and nested specs are parsed recursively.

Parameters

```
spec-parse(spec: any) -> RESULT(spec)
```

spec `any`

Spec shorthand.

to-string

Renders a spec as a compact string.

Shorthand specs are parsed first, so builtin types such as `int` work directly.

Parameters

```
to-string(
  spec: any,
  prec: int,
  depth: int
) -> str
```

spec `any`

Spec or spec shorthand.

prec `int`

Reserved precedence parameter.

Default: `0`

depth `int`

Recursive self depth.

Default: `0`